

EFREI
Master 1 CSM
Année 2026
Atelier sur Vagrant

1- Définition

a - Vagrant

Vagrant est un outil open-source qui simplifie la gestion des environnements de développement en automatisant la configuration des machines virtuelles. Il est principalement utilisé pour créer et gérer des environnements de développement reproductibles, ce qui facilite la collaboration entre les membres de l'équipe travaillant sur un projet.

Voici quelques caractéristiques clés de Vagrant :

1. **Gestion des Machines Virtuelles** : Vagrant permet de gérer des machines virtuelles en utilisant différentes technologies de virtualisation telles que VirtualBox, VMware, Hyper-V, etc. Cela signifie que les environnements de développement peuvent être créés et exécutés sur différentes plates-formes.
2. **Configuration Déclarative** : Les configurations de machines virtuelles sont déclarées dans un fichier texte appelé **Vagrantfile**. Ce fichier contient des informations telles que la configuration matérielle, le système d'exploitation, les scripts de provisionnement, etc.
3. **Provisionnement Automatisé** : Vagrant prend en charge le provisionnement automatique des machines virtuelles. Cela signifie que vous pouvez automatiser l'installation de logiciels, la configuration du réseau, et d'autres tâches grâce à des outils comme Shell, Ansible, Chef, ou Puppet.
4. **Environnements Reproductibles** : Les configurations Vagrant permettent aux développeurs de définir précisément les paramètres de leur environnement de développement. Cela garantit que tous les membres de l'équipe travaillent dans un environnement identique, réduisant ainsi les problèmes liés aux différences entre les environnements.

5. **Snapshots et Reprise d'État** : Vagrant prend en charge la création de snapshots pour les machines virtuelles, ce qui permet de revenir à un état précédent si nécessaire. Cela facilite le débogage et la gestion des changements.
6. **Intégration avec les Outils de Gestion de Configuration** : Vagrant peut être intégré avec des outils de gestion de configuration tels qu'Ansible, Chef, et Puppet pour automatiser davantage la configuration des machines virtuelles.
7. **Boîtes Vagrant (Vagrant Boxes)** : Les boîtes Vagrant sont des images de base préconfigurées qui peuvent être utilisées comme point de départ pour la création de machines virtuelles. Il existe de nombreuses boîtes Vagrant disponibles publiquement.
8. **Communauté Active** : Vagrant bénéficie d'une communauté active qui partage des boîtes, des scripts de provisionnement, et fournit un support sur les forums.

b- Mise en œuvre

Pour la mise en œuvre vous pouvez utiliser ce lien pour installer Vagrant suivant le système d'exploitation sur votre machine :

<https://developer.hashicorp.com/vagrant/install>

NB : Nous ferons l'installation sur Linux et Windows

2-1 Installation sur Windows :

Pour commencer, vous devez installer Vagrant sur votre ordinateur. Vous pouvez télécharger la dernière version de Vagrant à partir de son site web officiel (**<https://www.vagrantup.com/downloads>**).

Ou visiter ce lien pour avoir une version compatible avec votre système :

<https://releases.hashicorp.com/vagrant/>

Une fois téléchargé, exécutez la commande `vagrant --version` pour vérifier la version installée :

```
C:\Users\eldio>vagrant --version
Vagrant 2.4.1

C:\Users\eldio>|
```

La création d'une boîte de base est en fait spécifique au fournisseur. Cela signifie que selon que vous utilisez VirtualBox, VMware, AWS, etc...

On va ajouter notre première Base Boîte. Pour ce faire, nous allons sur notre terminal, ajouter un nouveau dossier nommé **efrei-vagrant**. On se déplace dans ce répertoire et on utilise la commande **vagrant box add centos/8** pour ajouter la box centos 8.

```
C:\Users\eldio\Downloads>mkdir efrei-vagrant
C:\Users\eldio\Downloads>cd efrei-vagrant
C:\Users\eldio\Downloads\efrei-vagrant>vagrant box add centos/8
==> box: Loading metadata for box 'centos/8'
      box: URL: https://vagrantcloud.com/api/v2/vagrant/centos/8
      This box can work with multiple providers! The providers that it
      can work with are listed below. Please review the list and choose
      the provider you will be working with.

1) libvirt
2) virtualbox

Enter your choice: 2
==> box: Adding box 'centos/8' (v2011.0) for provider: virtualbox
      box: Downloading: https://vagrantcloud.com/centos/boxes/8/versions/2011.0/providers/virtualbox/unknown/vagrant.box
      Download redirected to host: cloud.centos.org
      box:
      box: Calculating and comparing box checksum...
==> box: Successfully added box 'centos/8' (v2011.0) for 'virtualbox'!
C:\Users\eldio\Downloads\efrei-vagrant>
```

Une fois la box ajoutée nous pouvons lister les box avec la commande **vagrant box list**

```
C:\Users\eldio\Downloads\efrei-vagrant>vagrant box list
centos/8 (libvirt, 2011.0)
centos/8 (virtualbox, 2011.0) ✕

C:\Users\eldio\Downloads\efrei-vagrant>
```

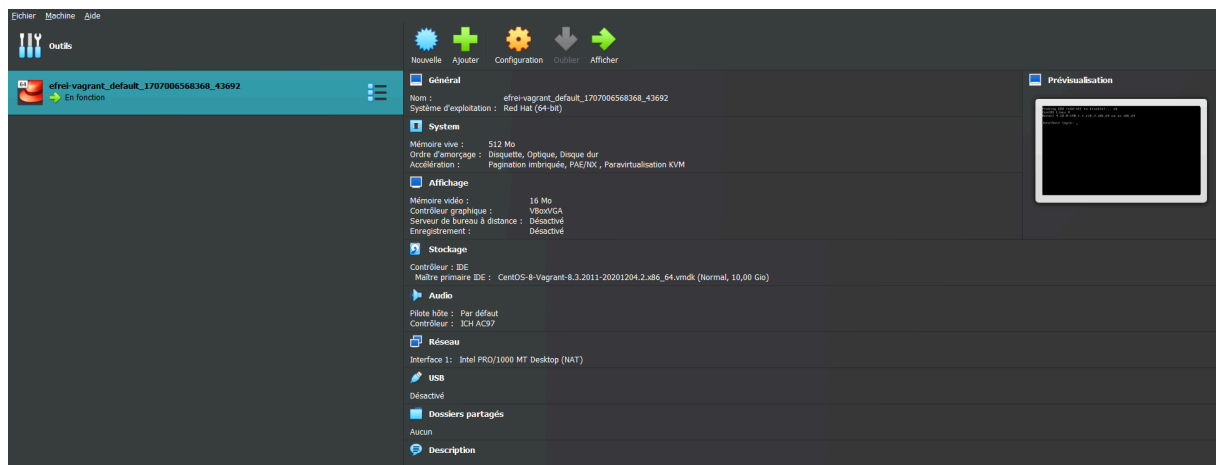
Ensuite nous allons passer à l'initialisation et au lancement de notre machine virtuelle en utilisant la commande **vagrant-up**

```

C:\Users\eldio\Downloads\efrei-vagrant>vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Importing base box 'centos/8'...
==> default: Matching MAC address for NAT networking...
==> default: Checking if box 'centos/8' version '2011.0' is up to date...
==> default: Setting the name of the VM: efrei-vagrant_default_1707006568368_43692
==> default: Clearing any previously set network interfaces...
==> default: Preparing network interfaces based on configuration...
default: Adapter 1: nat
==> default: Forwarding ports...
default: 22 (guest) => 2222 (host) (adapter 1)
==> default: Booting VM...
==> default: Waiting for machine to boot. This may take a few minutes...
default: SSH address: 127.0.0.1:2222
default: SSH username: vagrant
default: SSH auth method: private key
default:
default: Vagrant insecure key detected. Vagrant will automatically replace
default: this with a newly generated keypair for better security.
default:
default: Inserting generated public key within guest...
==> default: Machine booted and ready!
==> default: Checking for guest additions in VM...
default: No guest additions were detected on the base box for this VM! Guest
default: additions are required for forwarded ports, shared folders, host only
default: networking, and more. If SSH fails on this machine, please install
default: the guest additions and repack the box to continue.
default:
default: This is not an error message; everything may continue to work properly,
default: in which case you may ignore this message.
==> default: Rsyncing folder: /cygdrive/c/Users/eldio/Downloads/efrei-vagrant/ => /vagrant
C:\Users\eldio\Downloads\efrei-vagrant>

```

C'est done et nous allons vérifier si notre machine virtuelle est créée :



Parfait ! Notre machine virtuelle est bien créée.

Décommenter la ligne suivante pour éviter les mises à jour régulières :

config.vm.box_check_update = false

Installer le plugin vbguest

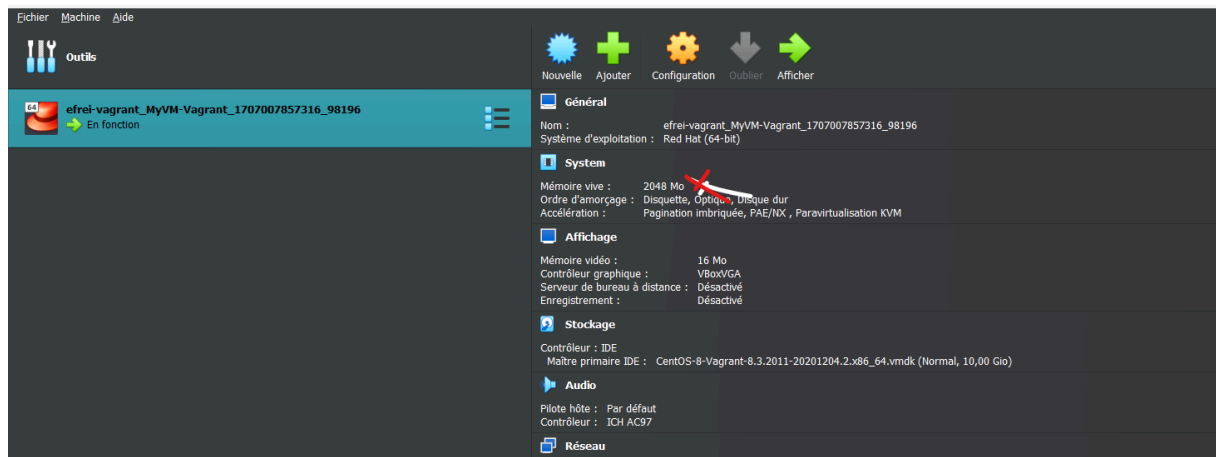
```

C:\Users\eldio\Downloads\efrei-vagrant>vagrant plugin install vagrant-vbguest
Installing the 'vagrant-vbguest' plugin. This can take a few minutes...
Fetching micromachine-3.0.0.gem
Fetching vagrant-vbguest-0.32.0.gem
Installed the plugin 'vagrant-vbguest (0.32.0)'!
C:\Users\eldio\Downloads\efrei-vagrant>

```

Défi 1 : Rechercher le fichier Vagrantfile :

- **Modifier le CPU à 2**
- **La mémoire à 2048 Mo**
- **Le nom de la Machine virtuelle (VM_votrenom)**



2-1-1 Les réseaux

Nous allons configurer le réseau public en modifiant la ligne suivante :

```
# Create a public network, which generally matched to bridged network.
# Bridged networks make the machine appear as another physical device on
# your network.
config.vm.network "public_network", bridge: "en0: Wi-fi (wireless)"

# Share an additional folder to the guest VM. The first argument is
# the path on the host to the actual folder. The second argument is
# the path on the guest to mount the folder. And the optional third
# argument is a set of non-required options.
```

Nous pouvons aussi configurer le réseau privé et faire le mappage de port

```
# accessing localhost:8080 will access port 80 on the guest machine.
# NOTE: This will enable public access to the opened port
config.vm.network "forwarded_port", guest: 80, host: 8080 ✗

# Create a forwarded port mapping which allows access to a specific port
# within the machine from a port on the host machine and only allow access
# via 127.0.0.1 to disable public access
# config.vm.network "forwarded_port", guest: 80, host: 8080, host_ip: "127.0.0.1"

# Create a private network, which allows host-only access to the machine
# using a specific IP.
config.vm.network "private_network", ip: "192.168.33.10" ✗
```

Enfin pour que la machine virtuelle prenne en compte les configurations faites sur Vagrantfile nous utilisons la commande **vagrant reload**

Une fois que la machine est bien configurée on se connecte via ssh avec la commande **vagrant ssh**

NB : Si vous avez des soucis pour télécharger les packages suivez les instructions ci-dessous.

```
[vagrant@myhost ~]$ sudo yum update
CentOS Linux 8 - AppStream                               123 B/s | 38 B   00:00
Error: Failed to download metadata for repo 'appstream': Cannot prepare internal mirrorlist: No URLs in mirrorlist
[vagrant@myhost ~]$ sudo sed -i 's/mirrorlist/#mirrorlist/g' /etc/yum.repos.d/CentOS-*
[vagrant@myhost ~]$ sudo sed -i 's|#baseurl=http://mirror.centos.org|baseurl=http://vault.centos.org|g' /etc/yum.repos.d/CentOS-*
[vagrant@myhost ~]$ sudo yum update
CentOS Linux 8 - AppStream                               4.7 MB/s | 8.4 MB 00:01
CentOS Linux 8 - BaseOS                                3.0 MB/s | 4.6 MB 00:01
CentOS Linux 8 - Extras                                9.2 kB/s | 10 kB  00:01
Dependencies resolved.
=====
Package                                Architecture      Version           Repository        Size
=====
Installing:
kernel                                x86_64            4.18.0-348.7.1.el8_5 baseos            7.0 M
kernel-core                           x86_64            4.18.0-348.7.1.el8_5 baseos            38 M
kernel-modules                         x86_64            4.18.0-348.7.1.el8_5 baseos            30 M
Upgrading:
NetworkManager                       x86_64            1:1.32.10-4.el8   baseos            2.6 M
NetworkManager-libnm                 x86_64            1:1.32.10-4.el8   baseos            1.8 M
NetworkManager-team                   x86_64            1:1.32.10-4.el8   baseos            148 k
NetworkManager-tui                   x86_64            1:1.32.10-4.el8   baseos            336 k
authselect                            x86_64            1.2.2-3.el8       baseos            133 k
authselect-libs                       x86_64            1.2.2-3.el8       baseos            222 k
avahi-libs                            x86_64            0.7-20.el8        baseos            62 k
bash                                  x86_64            4.4.20-2.el8      baseos            1.5 M
bind-export-libs                      x86_64            32:9.11.26-6.el8   baseos            1.1 M
brotli                                x86_64            1.0.6-3.el8       baseos            323 k
ca-certificates                       noarch            2021.2.50-80.0.el8_4 baseos            390 k
centos-gpg-keys                       noarch            1:8-3.el8         baseos            12 k
```

sudo sed -i 's/mirrorlist/#mirrorlist/g' /etc/yum.repos.d/CentOS-*

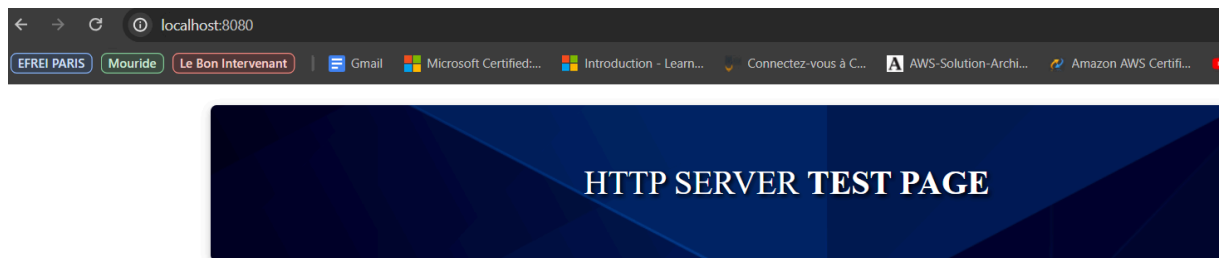
sudo sed -i 's|#baseurl=http://mirror.centos.org|baseurl=http://vault.centos.org|g' /etc/yum.repos.d/CentOS-*

sudo yum update -y

Vous pouvez jouer en installant d'autres services comme **httpd** et accéder à la page web.

```
[vagrant@myhost ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; vendor preset: disabled)
   Active: inactive (dead)
     Docs: man:httpd.service(8)
[vagrant@myhost ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[vagrant@myhost ~]$ sudo systemctl start httpd
[vagrant@myhost ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; vendor preset: disabled)
   Active: active (running) since Sun 2024-02-04 02:05:11 UTC; 2s ago
     Docs: man:httpd.service(8)
  Main PID: 31270 (httpd)
    Status: "Started, listening on: port 80"
     Tasks: 213 (limit: 12423)
    Memory: 26.9M
    CGroup: /system.slice/httpd.service
            └─31270 /usr/sbin/httpd -DFOREGROUND
              └─31271 /usr/sbin/httpd -DFOREGROUND
                └─31272 /usr/sbin/httpd -DFOREGROUND
                  └─31273 /usr/sbin/httpd -DFOREGROUND
                    └─31274 /usr/sbin/httpd -DFOREGROUND

Feb 04 02:05:11 myhost systemd[1]: Starting The Apache HTTP Server...
Feb 04 02:05:11 myhost httpd[31270]: AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive dynamically to determine the server's name. See the documentation for more details.
Feb 04 02:05:11 myhost systemd[1]: Started The Apache HTTP Server.
Feb 04 02:05:11 myhost httpd[31270]: Server configured, listening on: port 80
lines 1-19/19 (END)
```



This page is used to test the proper operation of the HTTP server after it has been installed. If you can read this page it means the server is working properly. This server is powered by [CentOS](#).

2-2 Installation sur Linux :

sudo apt update && sudo apt install vagrant

```
lamine@ELDIOP0:~/ssh$ sudo apt update && sudo apt install vagrant
Hit:1 https://apt.releases.hashicorp.com jammy InRelease
Hit:2 https://download.docker.com/linux/ubuntu jammy InRelease
Hit:3 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:4 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:5 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:6 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

sudo apt update

```
lamine@ELDIOP0:~/.ssh$ sudo apt update
[sudo] password for lamine:
Get:1 http://security.ubuntu.com/ubuntu jammy-security InRelease [110 kB]
Hit:2 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:3 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [119 kB]
Hit:4 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:5 https://apt.releases.hashicorp.com jammy InRelease
Hit:6 https://download.docker.com/linux/ubuntu jammy InRelease
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [1325 kB]
Get:8 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1042 kB]
Fetched 2596 kB in 1s (2487 kB/s)
Reading package lists... Done
```

sudo apt install qemu-kvm libvirt-daemon-system ksmtuned libvirt-clients libxslt-dev libxml2-dev libvirt-dev zlib1g-dev ruby-dev ruby-libvirt ebtables

```
lamine@ELDIOP0:~/.ssh$ sudo apt install qemu-kvm libvirt-daemon-system ksmtuned libvirt-clients libxslt-dev libxml2-dev libvirt-dev zlib1g-dev r
uby-dev ruby-libvirt ebtables
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Note, selecting 'qemu-system-x86' instead of 'qemu-kvm'
Note, selecting 'libxslt1-dev' instead of 'libxslt-dev'
ruby-dev is already the newest version (1:3.0~exp1).
ruby-dev set to manually installed.
The following additional packages will be installed:
  acl cpu-checker dmeventd dns-root-data dnsmasq-base glib-networking glib-networking-common glib-networking-services
  gstreamer1.0-plugins-base gstreamer1.0-plugins-good gstreamer1.0-x ibverbs-providers icu-devtools ipxe-qemu ipxe-qemu-256k-compat-efi-roms
  libaa1 libaio1 libasyncns0 libavc1394-0 libboost-iostreams1.74.0 libboost-thread1.74.0 libbrlapi0.8 libcacat0 libccard0 libcdparanoia0
  libdaxctl1 libdecor-0 libdecor-0-plugin-1-cairo libdevmapper-event1.02.1 libfdt1 libgfbapi0 libgfrpc0 libgfxdr0 libglusterfs0
  libgstreamer-plugins-base1.0-0 libgstreamer-plugins-good1.0-0 libgudev-1.0-0 libibverbs1 libicu-dev libiec61883-0 libiscsi7
  libjson-glib-1.0-0 libjson-glib-1.0-common liblm2cmd2.03 libmpg123-0 libndctl6 libnl-route-3-200 libnss-mymachines libnss-systemd
  liborc-0.4-0 libpam-systemd libpmem1 libpmemobj1 libproxy1v5 libpulse0 librados2 libraw1394-11 librbdl1 librdmacm1 libsd12-2.0-0 libshout3
  libsndfile1 libsoup2.4-1 libsoup2.4-common libspice-server1 libsystemd0 libtag1v5 libtag1v5-vanilla libtpms0 libusbredirparser1
  libvirglrenderer1 libvirt-daemon libvirt-daemon-config-network libvirt-daemon-config-nwfilter libvirt-daemon-driver-qemu
  libvirt-daemon-system libvirt0 libvisual-0.4-0 libwavpack1 libxen-dev libxencall1 libxendevicemodel1 libxenevtchn1
```

sudo usermod -aG libvirt \$USER

```
lamine@ELDIOP0:~/.ssh$ sudo usermod -aG libvirt $USER
```

vagrant plugin install vagrant-libvirt

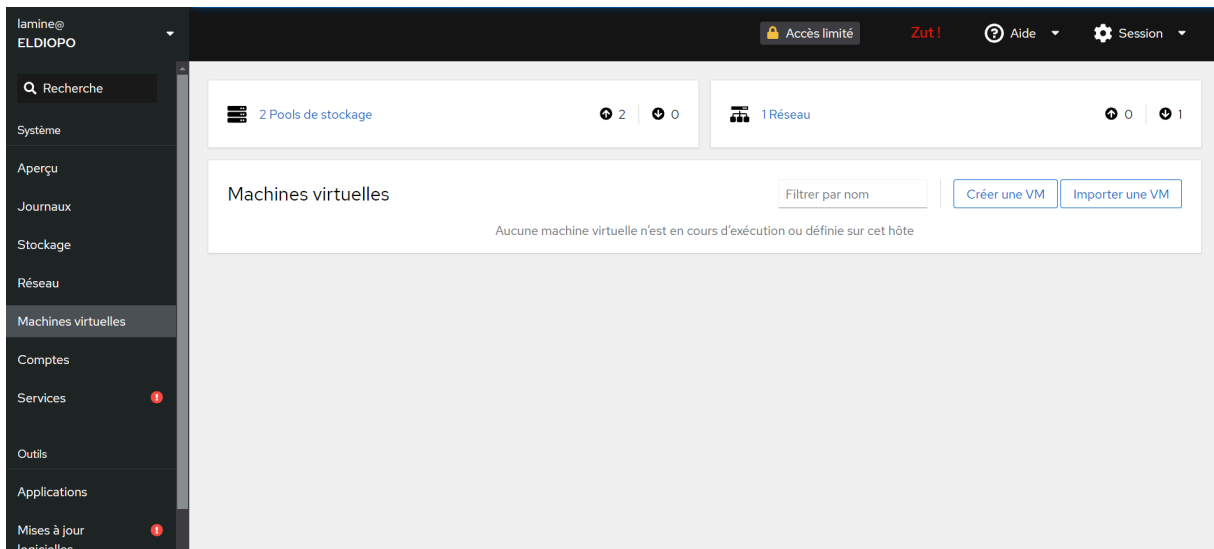
sudo apt update

```
lamine@ELDIOP0:~/.ssh$ vagrant plugin install vagrant-libvirt
Installing the 'vagrant-libvirt' plugin. This can take a few minutes...
Fetching xml-simple-1.1.9.gem
Fetching nokogiri-1.16.0-x86_64-linux.gem
Fetching ruby-libvirt-0.8.0.gem
Building native extensions. This could take a while...
Fetching formatador-1.1.0.gem
Fetching fog-core-2.4.0.gem
Fetching fog-xml-0.1.4.gem
Fetching fog-json-1.2.0.gem
Fetching fog-libvirt-0.12.0.gem
Fetching diffy-3.4.2.gem
Fetching vagrant-libvirt-0.12.2.gem
Installed the plugin 'vagrant-libvirt (0.12.2)'!
lamine@ELDIOP0:~/.ssh$ sudo apt update
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 https://download.docker.com/linux/ubuntu jammy InRelease
Hit:3 https://apt.releases.hashicorp.com jammy InRelease
Hit:4 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:5 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:6 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

sudo apt install cockpit cockpit-machines

```
lamine@ELDIOP0:~/.ssh$ sudo apt install cockpit cockpit-machines
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  busybox-initramfs cockpit-bridge cockpit-networkmanager cockpit-packagekit cockpit-storaged cockpit-system cockpit-ws cracklib-runtime
  finalrd gdisk girl1.2-freedesktop girl1.2-libosinfo-1.0 initramfs-tools-bin initramfs-tools-core klibc-utils libatasmart4 libblockdev-
  libblockdev-fs2 libblockdev-loop2 libblockdev-mdraid2 libblockdev-part-err2 libblockdev-part2 libblockdev-swap2 libblockdev-utils2
  libblockdev2 libbluetooth3 libburn4 libbytesize-common libbytesize1 libcrack2 libfdisk1 libgovirt-common libgovirt2 libgpgme11
  libgtk-vnc-2.0-0 libgvnc-1.0-0 libisoburn1 libisofs6 libjte2 libklibc libmbim-glib4 libmbim-proxy libmm-glib0 libndp0 libnm0 libosinfo-
  libparted-fs-resize0 libphodav-2.0-0 libphodav-2.0-common libpwquality-common libpwquality-tools libpwquality1 libqmi-glib5 libqmi-p
  librest-0.7-0 libsoup-gnome2.4-1 libspice-client-glib-2.0-8 libspice-client-gtk-3.0-5 libteamdctl0 libudisks2-0 libusbredirhost1
  libvirt-dbus libvirt-glib-1.0-0 libvirt-glib-1.0-data libvolume-key1 mdadm modemmanager network-manager network-manager-pptp osinfo-
  pptp-linux python3-libvirt python3-libxml2 spice-client-glib-usb-acl-helper udisks2 usb-modeswitch usb-modeswitch-data virt-viewer v
  wamerican wireless-regdb wpasupplicant xorriso zstd
Suggested packages:
  cockpit-doc cockpit-pcp cockpit-sosreport udisks2-lvm2 sssd-dbus libosinfo-l10n libparted-dev gstreamer1.0-libav gstreamer1.0-plugin
  default-mta | mail-transport-agent dracut-core avahi-autoipd libteam-utils btrfs-progs f2fs-tools nilfs-tools reiserfsprogs udftools
  udisks2-bcache udisks2-btrfs udisks2-zram xfsprogs exfatprogs comgt wvdial python3-argcomplete wpagui libengine-pkcs11-openssl xorri
  jiqit cdck
```

Ensuite pour gérer vos vm cliquez sur ce lien <http://localhost:9090>. Pour vous connecter, entrez le user de votre machine virtuelle et le mdp. La gestion des machines virtuelles se retrouve dans la section du même nom.



Depuis l'interface nous remarquons que nous pouvons :

- Créer des machines virtuelles
- Importer des machines virtuelles
- Gérer notre réseau et le stockage.

Essayons maintenant de créer une machine virtuelle depuis l'interface. Nous allons tout simplement cliquer sur créer une machine virtuelle,

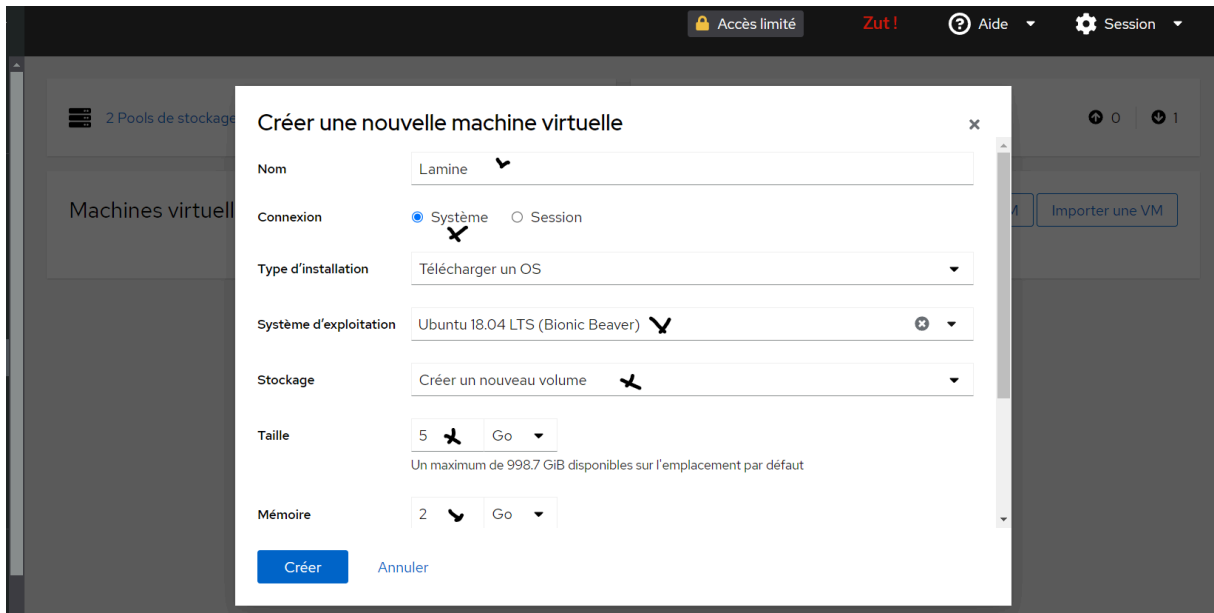
Nom de VM : Lamine

Système d'exploitation : Ubuntu 18.04 LTS

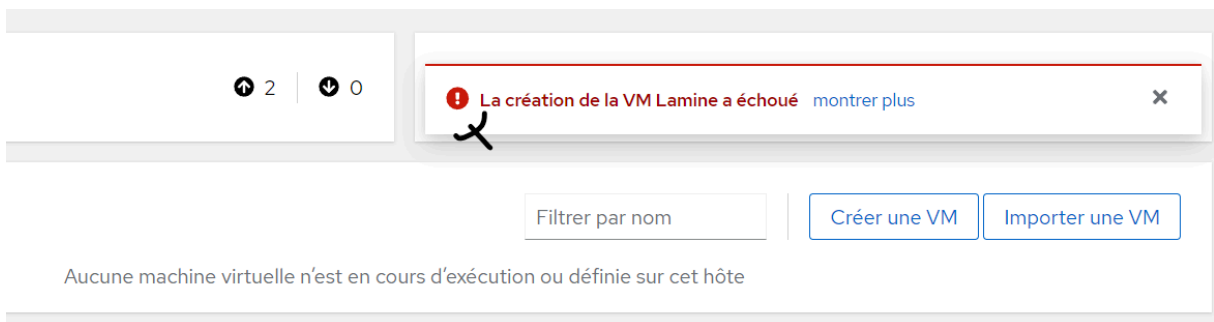
Volume créé : Stockage 5 Go

Mémoire : 2 Go

Une fois ces paramètres validés nous allons cliquer sur Créer.

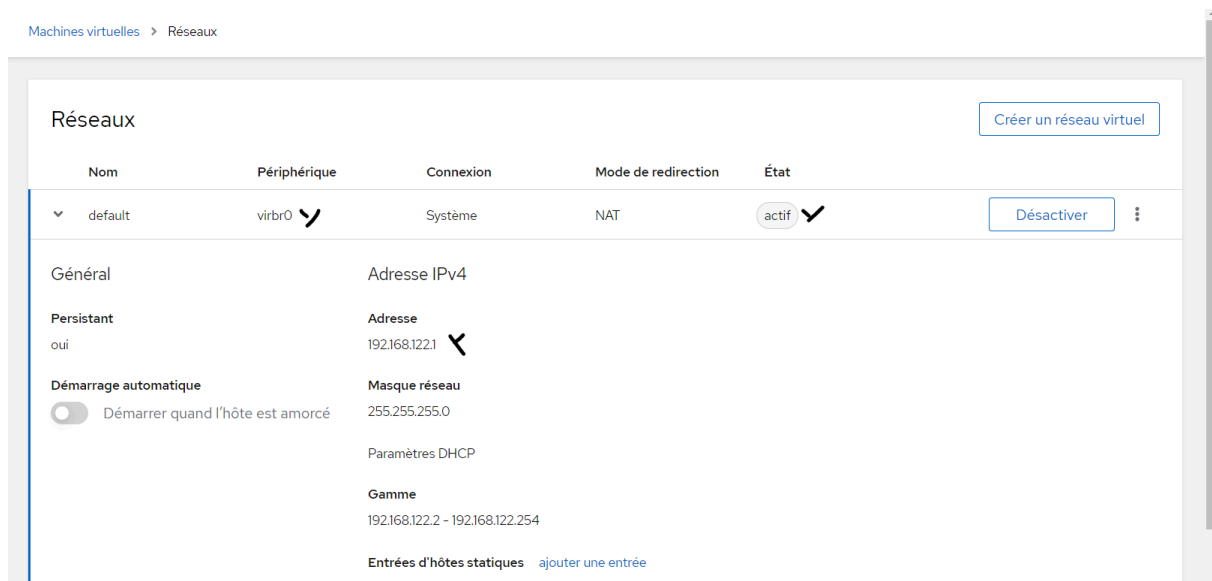


La nous avons une erreur car le réseau n'est pas configuré :

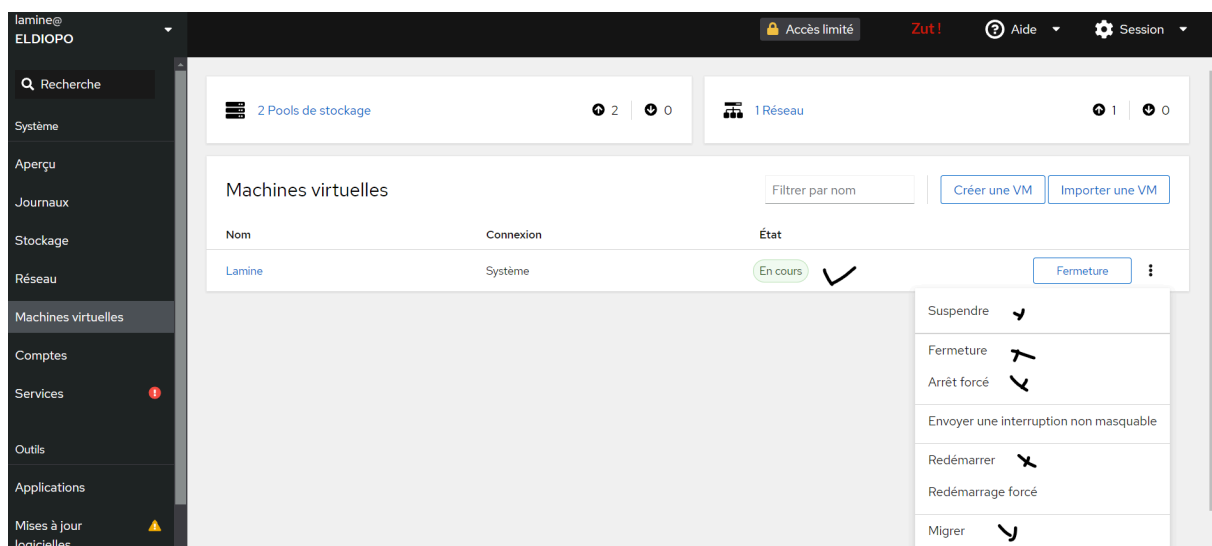


Défi 2 : Troubeshooter et faite une configuration réseau pour réussir la création de la machine virtuelle.

De mon côté j'ai réussi à créer un réseau



En jetant un coup d'œil sur l'onglet machine virtuelle je vois que ma machine virtuelle est en cours d'exécution



Je peux voir les détails de ma machine virtuelle

- Réseaux
- Disque
- Instantanés
- Interfaces

lamine@ELDIOPO

Recherche

Systeme

Aperçu

Journaux

Stockage

Reseau

Machines virtuelles

Comptes

Services

Outils

Applications

Mises à jour

Accès limitéZut!AideSession

Utilisation

Mémoire31 / 3 GiB

CPU617% of 2 vCPUs

Disques

Ajouter un disque

Périp...	Utilisé	Capacité	Bus	Accès	La source	Addition...
disk	10 Go	10 Go	virtio	Mode écriture	Fichier	/var/lib/libvirt/images/Lamine.qcow2

RetirerModifier

Interfaces réseau

Ajouter une interface réseau

Type	Type de mod...	Adresse MAC	Adresse IP	La source	État	
network	virtio	52:54:00:cc:2e:94	inet 192.168.122.114/24	default	en cours	

SupprimerUnplugModifier